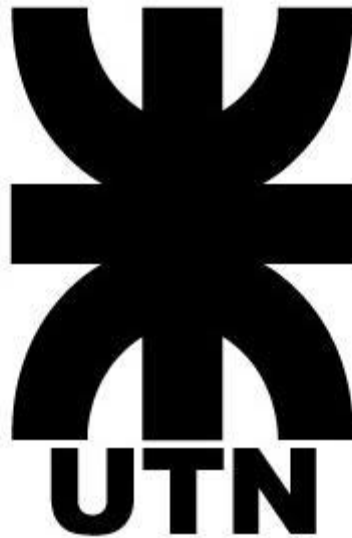


Universidad Tecnológica Nacional



Cátedra: Algoritmos Genéticos

4º Año Ingeniería en Sistemas de Información

Trabajo Práctico N°2

Comisión: 303

Grupo : 14

Integrantes:

- RIBOTTA, Tomás	52309	tomasribotta20@gmail.com
- MAZALAN, Ariel	52867	arielmazalan11@gmail.com
- GIACONE, Alessandro	52664	alef.giacone@gmail.com
- BUSNADIEGO, Martin	53020	martinbusnaisi@gmail.com

1. Introducción Teórica	2
1.1. Optimización Combinatoria	2
1.2. El Problema de la Mochila	3
1.3. Formalización Matemática	3
1.4. Complejidad del Problema	4
2. Métodos de Resolución	4
2.1. Búsqueda Exhaustiva	4
2.2. Algoritmo Heurístico Greedy (Goloso)	5
2.3. Comparación entre Enfoques	5
3. Descripción del Problema	6
3.1. Datos: Ejercicios 1 y 2	6
3.2. Datos: Ejercicio 3	6
4. Ejercicio 1: Búsqueda Exhaustiva (10 objetos)	7
4.1. Representación del espacio de soluciones	7
4.2. Codificación en Python	7
4.3. Resultados	7
5. Ejercicio 2: Algoritmo Greedy (10 objetos)	8
5.1. Cálculo del ratio valor/volumen	8
5.2. Codificación en Python	8
5.3. Resultados	8
5.4. Comparación con la Búsqueda Exhaustiva	9
6. Ejercicio 3: Problema de la Mochila con Restricción de Peso	9
6.1. Resolución por Algoritmo Goloso	9
6.2. Resolución Exhaustiva	9
6.3. Análisis Comparativo y Conclusiones del Ejercicio	10
7. Conclusiones Generales	10
8. Anexos: Código Fuente en Python	10
8.1. Ejercicio 1 — Búsqueda Exhaustiva	10
8.2. Ejercicio 2 — Algoritmo Greedy	10
8.3. Ejercicio 3 — Búsqueda Exhaustiva y Greedy (3 elementos)	10

1. Introducción Teórica

1.1. Optimización Combinatoria

A pesar del desarrollo científico de los últimos años, aún quedan muchos problemas reales de optimización con una gran importancia práctica que no han sido resueltos. Dentro de este grupo están los que se caracterizan porque sus espacios solución están formados por subconjuntos de números naturales y, por dicha razón, se los llama **problemas de optimización combinatoria**.

La optimización combinatoria es una rama de la optimización en matemáticas aplicadas y en ciencias de la computación, relacionada con la investigación de operaciones, la teoría de algoritmos y la teoría de la complejidad computacional. También está relacionada con otros campos como la inteligencia artificial e ingeniería de software.

Los algoritmos de optimización combinatoria resuelven instancias de problemas que se creen difíciles en general, explorando el espacio de soluciones (usualmente grande) para estas instancias. Lo logran reduciendo el tamaño efectivo del espacio y explorando el espacio de búsqueda eficientemente.

Quizás los dos problemas de optimización combinatoria más famosos son el problema de la mochila y el problema del viajante.

1.2. El Problema de la Mochila

El problema de la mochila consiste en elegir, de entre un conjunto de **N** elementos (cada uno con un valor $\$i$ y un volumen V_i), aquellos que puedan ser cargados en una mochila de volumen **V**, de manera que el valor total obtenido sea máximo.

El conjunto solución debe respetar la restricción de capacidad:

$$\sum_{i \in S} V_i \leq V$$

y maximizar:

$$\sum_{i \in S} \$i$$

1.3. Formalización Matemática

Dado un conjunto de **N** elementos, cada uno caracterizado por:

- V_i : volumen (o peso) del elemento i
- $\$i$: valor del elemento i

Y una mochila de capacidad máxima V , se busca el subconjunto $S \subseteq \{1, 2, \dots, N\}$ tal que:

$$\text{Maximizar } f(S) = \sum_{i \in S} \$i$$

$$\text{sujeto a } \sum_{i \in S} V_i \leq V$$

1.4. Complejidad del Problema

Planteando el problema de la mochila con un procedimiento exhaustivo que evalúa todos los subconjuntos posibles, la solución encontrada tendría la importancia de ser única (óptima global). Sin embargo, el número de subconjuntos a evaluar crece exponencialmente con N .

Si el conjunto es de N elementos, el espacio solución tendrá 2^N posibles subconjuntos. Por ejemplo:

N (elementos)	2^N (subconjuntos)	Tiempo estimado (1M eval/seg)
10	1.024	Milisegundos
20	1.048.576	~1 segundo
40	$\sim 1,1 \times 10^{12}$	~2 semanas
60	$\sim 1,15 \times 10^{18}$	~365 siglos

Esto demuestra que **el problema es manejable cuando N es reducido**, pero a medida que N crece el tiempo aumenta rápidamente, y para valores grandes de N no hay forma de resolverlo exhaustivamente en tiempo razonable.

2. Métodos de Resolución

2.1. Búsqueda Exhaustiva

La búsqueda exhaustiva consiste en evaluar **todos los subconjuntos posibles** de elementos, calcular para cada uno el valor total y el volumen total, descartar aquellos que superen la capacidad de la mochila y seleccionar el subconjunto de mayor valor entre los válidos.

Cada subconjunto puede representarse como una cadena binaria de N bits, donde un 1 en la posición i indica que el elemento i fue incluido. De este modo se enumeran los 2^N subconjuntos iterando de 0 a $2^N - 1$.

Este método **garantiza encontrar la solución óptima**, pero su costo computacional lo hace inviable para valores grandes de N .

2.2. Algoritmo Heurístico Greedy (Goloso)

Cuando el problema es necesario resolverlo inmediatamente o cuando N es grande, se recurre a métodos **heurísticos**: algoritmos que no garantizan la solución óptima, pero que encuentran soluciones factibles (que respetan las restricciones) de buena calidad en un tiempo de cálculo razonable.

Una clase particular de heurísticas son los **métodos constructivos**, que agregan componentes individuales a la solución hasta que se obtiene una solución factible.

El **algoritmo greedy** (goloso o devorador) es un ejemplo de estos métodos. Construye la solución paso a paso eligiendo, en cada paso, la opción que ofrece el **máximo beneficio inmediato**.

Para el problema de la mochila, el criterio greedy consiste en:

1. Calcular el ratio $\$i / Vi$ (valor por unidad de volumen) para cada elemento.
2. Ordenar los elementos de mayor a menor ratio.
3. Cargar la mochila en ese orden hasta que no entre ningún elemento más.
4. Si al detenerse, algún elemento posterior (de menor ratio) cabe en el espacio restante, incluirlo.

Importante: el algoritmo greedy **no garantiza la solución óptima global**. En algunos casos puede diferir del resultado exhaustivo, como se analizará en el Ejercicio 3.

2.3. Comparación entre Enfoques

Característica	Búsqueda Exhaustiva	Algoritmo Greedy
Garantía de optimalidad	Sí	No

Característica	Búsqueda Exhaustiva	Algoritmo Greedy
Complejidad temporal	$O(2^N)$	$O(N \log N)$
Aplicable para N grande	No	Sí
Facilidad de implementación	Media	Alta

3. Descripción del Problema

3.1. Datos: Ejercicios 1 y 2

Se dispone de 10 objetos con sus respectivos volúmenes y valores, y una mochila de capacidad máxima **4200 cm³**:

Objeto	Volumen (cm³)	Valor (\$)
1	150	20
2	325	40
3	600	50
4	805	36
5	430	25
6	1200	64
7	770	54
8	60	18
9	930	46
10	353	28

Volumen máximo soportado por la mochila: 4200 cm³

3.2. Datos: Ejercicio 3

Se trabaja con 3 elementos con restricción de **peso** (en lugar de volumen) y una mochila de capacidad **3000 grs**:

Elemento	Peso (grs)	Valor (\$)
1	1800	72
2	600	36
3	1200	60

Capacidad máxima de la mochila: 3000 grs

4. Ejercicio 1: Búsqueda Exhaustiva (10 objetos)

4.1. Representación del espacio de soluciones

Con $N = 10$ objetos, el espacio de soluciones está compuesto por $2^{10} = 1024$ **subconjuntos** posibles. Cada subconjunto se representa como un número binario de 10 bits: si el bit i es 1, el objeto i se incluye en la mochila; si es 0, no se incluye.

Para cada subconjunto se evalúa:

- **Volumen total**: suma de los volúmenes de los objetos incluidos.
- **Valor total**: suma de los valores de los objetos incluidos.

Un subconjunto es **válido** si su volumen total no supera los 4200 cm³. Entre todos los válidos, se selecciona el de mayor valor.

4.2. Codificación en Python

[COMPLETAR: agregar el código Python aquí una vez implementado]

4.3. Resultados

[COMPLETAR: agregar tabla con el/los mejores subconjuntos encontrados, volumen total y valor total]

5. Ejercicio 2: Algoritmo Greedy (10 objetos)

5.1. Cálculo del ratio valor/volumen

El primer paso del algoritmo greedy es calcular el ratio $\$i / Vi$ para cada objeto y ordenarlos de mayor a menor:

Objeto	Volumen (cm³)	Valor (\$)	Ratio (\$/cm³)
8	60	18	[COMPLETAR]
1	150	20	[COMPLETAR]
2	325	40	[COMPLETAR]
7	770	54	[COMPLETAR]
3	600	50	[COMPLETAR]
6	1200	64	[COMPLETAR]
9	930	46	[COMPLETAR]
10	353	28	[COMPLETAR]
5	430	25	[COMPLETAR]
4	805	36	[COMPLETAR]

Nota: la tabla anterior está ordenada de forma tentativa; los valores de ratio se calcularán con el código.

5.2. Codificación en Python

[COMPLETAR: agregar el código Python aquí una vez implementado]

5.3. Resultados

[COMPLETAR: agregar los objetos seleccionados por greedy, volumen total y valor total]

5.4. Comparación con la Búsqueda Exhaustiva

[COMPLETAR: analizar si greedy alcanzó el óptimo o no, comentar similitudes y diferencias con el resultado exhaustivo]

6. Ejercicio 3: Problema de la Mochila con Restricción de Peso

6.1. Resolución por Algoritmo Goloso

[COMPLETAR: mostrar el cálculo de ratios $\$i/P_i$, el orden resultante y los elementos seleccionados paso a paso]

6.2. Resolución Exhaustiva

Con $N = 3$ elementos, el espacio de soluciones tiene $2^3 = 8$ subconjuntos posibles, perfectamente enumerables a mano:

Subconjunto	Elementos	Peso total (grs)	Valor total (\$)	¿Válido?
000	Ninguno	0	0	✓
001	{3}	1200	60	✓
010	{2}	600	36	✓
011	{2,3}	1800	96	✓
100	{1}	1800	72	✓
101	{1,3}	3000	132	✓

Subconjunto	Elementos	Peso total (grs)	Valor total (\$)	¿Válido?
110	{1,2}	2400	108	✓
111	{1,2,3}	3600	168	✗

[COMPLETAR: indicar cuál es el subconjunto óptimo]

6.3. Análisis Comparativo y Conclusiones del Ejercicio

[COMPLETAR: comparar el resultado greedy vs exhaustivo para este caso. Analizar si coinciden o no, y por qué. Esta es la parte clave del ejercicio: el greedy puede NO encontrar la solución óptima en ciertos casos, y aquí se debe evidenciar y explicar]

7. Conclusiones Generales

[COMPLETAR: sintetizar las conclusiones de los tres ejercicios. Algunas ideas orientadoras:]

- La búsqueda exhaustiva garantiza la solución óptima pero su complejidad $O(2^N)$ la hace inviable para N grande.
- El algoritmo greedy ofrece soluciones de calidad en tiempo $O(N \log N)$, pero no garantiza el óptimo global.
- El Ejercicio 3 demuestra concretamente un caso donde greedy falla en encontrar el óptimo.
- Para N pequeño (como los casos estudiados), la búsqueda exhaustiva es la elección natural.
- Para N grande, los métodos heurísticos (como los Algoritmos Genéticos estudiados en el TP anterior) se vuelven necesarios.

8. Anexos: Código Fuente en Python

8.1. Ejercicio 1 — Búsqueda Exhaustiva

[COMPLETAR]

8.2. Ejercicio 2 — Algoritmo Greedy

[COMPLETAR]

8.3. Ejercicio 3 — Búsqueda Exhaustiva y Greedy (3 elementos)

[COMPLETAR]